Pushkar Gudla
Email: ai24btech11012@iith.ac.in
Course: AI3403 - Multi-Agent Systems (MAS)
Instructor: Dr. Venkatraman Renganathan

Assignment - 5
ID: AI24BTECH11012
Term: Fall 2026
Due Date: 30th October, 2026

Problem 1:
Cooperative Navigation Problem (25 pts)

Consider a 5×5 grid with coordinates (x, y) where $x, y \in \{1, 2, 3, 4, 5\}$. Two agents (Agent 1 and Agent 2) move on the grid with actions $\mathcal{A}_i = \{\text{Up, Down, Left, Right, Stay}\}$, for $i \in \{1, 2\}$. Actions that would move an agent outside the grid leave it in place (wall). Let the positions of both the agents at time t be $p_{1,t} = (x_{1,t}, y_{1,t})$ and $p_{2,t} = (x_{2,t}, y_{2,t})$. There are two landmarks g_1 and g_2 represented using grid cells. An episode ends when both landmarks are covered, meaning that at time t, there exists a one-to-one assignment of agents to landmarks such that each landmark has one agent on it (agents can swap roles). Episodes also terminate at a fixed horizon $H = 30$ if not yet successful. At each step, the team reward is

$$r_t = \begin{cases} +20 - 1, & \text{if both landmarks become covered at time } t, \\ -5, & \text{collision penalty if } p_{1,t} = p_{2,t}, \\ -1, & \text{otherwise,} \end{cases}$$

1. (5 points) Formulate the above environment as a MAMDP and specify the tuple details along with the termination condition.
2. (5 points) Assume each agent runs Independent Q-Learning (IQL) with a tabular $Q_i(s, a_i)$ and ε-greedy exploration. Write the IQL update equation for Q_i, $i = \{1, 2\}$ and explain why the learning problem becomes non-stationary from the viewpoint of an individual agent, even though the environment dynamics are fixed.
3. (5 points) Explain the key idea of Centralized Training with Decentralized Execution (CTDE) in MARL. Describe mathematically a centralized action-value function $Q(s, a_1, a_2)$ and how it can be used during training. Give two concrete reasons why CTDE helps compared to purely independent learning in cooperative tasks.
4. (10 points – Coding Task) Implement Independent Q-Learning (IQL) for the two-agent gridworld. Choose S be a random positive integer less than 10^4. Define $k = \text{mod}(S, 5) + 1$. Define the target location as $g_1 = (1, k)$, $g_2 = (5, 6 - k)$ and let the initial positions of both the agents be $p_{1,0} = (1, 1)$, $p_{2,0} = (5, 5)$. Use $\alpha = 0.2$, $\gamma = 0.95$, $\varepsilon_0 = 1.0$, $\varepsilon_{\min} = 0.05$, $\varepsilon \leftarrow \max(\varepsilon_{\min}, 0.995\,\varepsilon)$ per episode. Train for $N_{ep} = 2000$ episodes with horizon $H = 30$. Your submission must include:
   - A plot of the moving-average episode return (window 50).
   - A plot of the moving-average success rate (window 50), where success means both landmarks covered.
   - Print greedy (no exploration) trajectories for 3 evaluation episodes after training from fixed initial positions, showing both agents' step-by-step positions.

Solution:

1. MAMDP formulation

The environment is the tuple $\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}, P, R, \gamma, s_0 \rangle$:

- **Agents:** $\mathcal{N} = \{1, 2\}$.
- **State:** $s_t = (p_{1,t}, p_{2,t})$ with $p_{i,t} \in \{1, \ldots, 5\}^2$, so $|\mathcal{S}| = 625$ (the landmarks g_1, g_2 are fixed parameters; the time index t is needed only for the horizon).
- **Actions:** $\mathcal{A}_i = \{\text{Up, Down, Left, Right, Stay}\}$, joint action $a_t = (a_{1,t}, a_{2,t}) \in \mathcal{A}_1 \times \mathcal{A}_2$.
- **Transition (deterministic):** $p_{i,t+1} = p_{i,t} + \delta(a_{i,t})$ if the result lies inside the grid, otherwise $p_{i,t+1} = p_{i,t}$.

- **Reward (shared):** let $C(s) = \mathbb{1}\big[(p_1, p_2) = (g_1, g_2) \text{ or } (p_1, p_2) = (g_2, g_1)\big]$. Then

$$r_t = \begin{cases} +20 - 1 = +19 & C(s_{t+1}) = 1, \\ -5 & p_{1,t+1} = p_{2,t+1}, \\ -1 & \text{otherwise.} \end{cases}$$

  ($C = 1$ and a collision cannot occur together since $g_1 \neq g_2$.)
- **Discount and start:** $\gamma = 0.95$, $s_0 = ((1, 1), (5, 5))$.
- **Termination:** the episode ends at the first t with $C(s_t) = 1$ (success), or at $t = H = 30$ (timeout).

Since both agents receive the same reward, the game is fully cooperative.

2. Independent Q-learning

Each agent i keeps a table $Q_i(s, a_i)$, acts ε-greedily ($a_i = \arg\max_a Q_i(s, a)$ w.p. $1-\varepsilon$, uniformly random otherwise), and after observing (s, a_1, a_2, r, s') updates

$$Q_i(s, a_i) \leftarrow Q_i(s, a_i) + \alpha\Big[r + \gamma\,(1 - \text{done}) \max_{a' \in \mathcal{A}_i} Q_i(s', a') - Q_i(s, a_i)\Big], \qquad i = 1, 2.$$

Non-stationarity. Agent i ignores the other agent's action, so it effectively sees the transition and reward

$$P_i(s'|s, a_i) = \sum_{a_j} \pi_j(a_j|s)\, P(s'|s, a_i, a_j), \qquad r_i(s, a_i) = \sum_{a_j} \pi_j(a_j|s)\, r(s, a_i, a_j).$$

Although P and r are fixed, π_j changes continually (Q-updates and the decay of ε), so P_i and r_i drift over time. Q-learning's convergence guarantee requires a stationary MDP, which therefore no longer holds, and the two agents' learning processes chase each other.

3. Centralized training, decentralized execution (CTDE)

Idea. During training, learning uses global information (state, joint action, other agents' policies); at execution, each agent acts only from its own observation, $a_i = \pi_i(o_i)$.

Centralized action-value.

$$Q(s, a_1, a_2) = \mathbb{E}\Big[\sum_{t \geq 0} \gamma^t r_t \;\Big|\; s_0 = s,\, a_0 = (a_1, a_2),\, \pi\Big],$$

learned with

$$Q(s, a_1, a_2) \leftarrow Q + \alpha\big[r + \gamma \max_{a_1', a_2'} Q(s', a_1', a_2') - Q\big].$$

It is used in training either (i) through a factorisation such as $Q \approx Q_1(o_1, a_1) + Q_2(o_2, a_2)$ (VDN/QMIX), so that the decentralised greedy actions $a_i = \arg\max Q_i$ are consistent with the joint maximiser, or (ii) as a critic for actor–critic methods, with policy gradient $\nabla_{\theta_i} J = \mathbb{E}\big[\nabla_{\theta_i} \log \pi_i(a_i|o_i)\, Q(s, a_1, a_2)\big]$.

Why it helps over independent learning.

1. *Stationarity:* $Q(s, a_1, a_2)$ conditions on the other agent's actual action, so its target does not change when the other agent's policy changes. This removes the non-stationarity of Part 2.
2. *Credit assignment and coordination:* with a shared reward, an agent cannot tell whether -5 (collision) or $+19$ came from its own action or its partner's. The joint value separates these cases and resolves role assignment (which agent takes which landmark).

4. Coding task (IQL)

Chosen $S = 7341 \Rightarrow k = \text{mod}(7341, 5) + 1 = 2$, so $g_1 = (1, 2)$, $g_2 = (5, 4)$. Training used $\alpha = 0.2$, $\gamma = 0.95$, $\varepsilon : 1.0 \to 0.05$ (decay 0.995/episode), $N_{ep} = 2000$, $H = 30$. Each agent's table is indexed by the joint state.

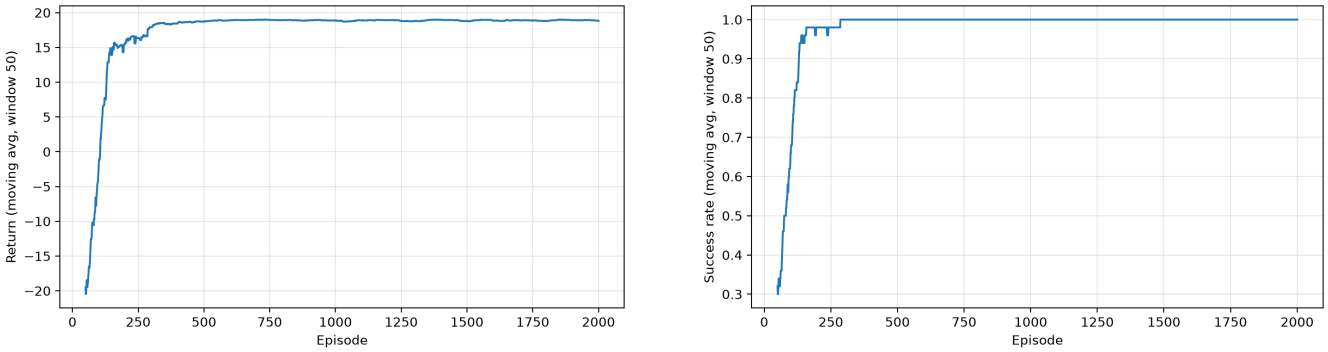


Figure 1: Moving average (window 50) of the episode return (left) and success rate (right).

Over the last 50 training episodes the success rate is 1.00. The three greedy evaluation episodes from $(1, 1), (5, 5)$ were identical:

t	Agent 1	Agent 2	r_t
0	$(1, 1)$	$(5, 5)$	–
1	$(1, 2)$ (Up)	$(5, 4)$ (Down)	+19

Table 1: Greedy trajectory (episodes 1–3): return = 19, success, 1 step.

Each landmark is one step from its agent's start cell, so the learned policy attains the optimal return of 19.

Code

```python
import os
import numpy as np
import matplotlib.pyplot as plt

OUT = os.path.dirname(os.path.abspath(__file__))
S = 7341                                    # your random integer < 10^4
k = S % 5 + 1
G1, G2 = (1, k), (5, 6 - k)
ALPHA, GAMMA, EPS_MIN, DECAY = 0.2, 0.95, 0.05, 0.995
N_EP, H, W = 2000, 30, 50
MOVES = {"Up": (0, 1), "Down": (0, -1), "Left": (-1, 0), "Right": (1, 0), "Stay": (0, 0)}
NAMES = list(MOVES)
rng = np.random.default_rng(0)


def move(p, a):
    x, y = p[0] + MOVES[NAMES[a]][0], p[1] + MOVES[NAMES[a]][1]
    return (x, y) if 1 <= x <= 5 and 1 <= y <= 5 else p       # wall: stay in place


def step(p1, p2, a1, a2):
    p1, p2 = move(p1, a1), move(p2, a2)
    if (p1, p2) in [(G1, G2), (G2, G1)]:
        return p1, p2, 19, True                                  # +20 - 1
    return p1, p2, (-5 if p1 == p2 else -1), False
```

```
26
27
28  def idx(p1, p2):                                                    # joint state -> 0..624
29      return ((p1[0] - 1) * 5 + p1[1] - 1) * 25 + (p2[0] - 1) * 5 + p2[1] - 1
30
31
32  # ---- IQL training: one Q-table per agent over the joint state ----
33  Q = np.zeros((2, 625, 5))
34  rets, succ, eps = [], [], 1.0
35  for ep in range(N_EP):
36      p1, p2, G, done = (1, 1), (5, 5), 0, False
37      for t in range(H):
38          s = idx(p1, p2)
39          a = [rng.integers(5) if rng.random() < eps else Q[i, s].argmax() for i in range
                (2)]
40          p1, p2, r, done = step(p1, p2, *a)
41          s2 = idx(p1, p2)
42          for i in range(2):
43              Q[i, s, a[i]] += ALPHA * (r + GAMMA * Q[i, s2].max() * (not done) - Q[i, s, a
                    [i]])
44          G += r
45          if done:
46              break
47      rets.append(G)
48      succ.append(float(done))
49      eps = max(EPS_MIN, DECAY * eps)
50
51  # ---- Plots ----
52  ma = lambda x: np.convolve(x, np.ones(W) / W, "valid")
53  for data, name, ylabel in [(rets, "return", "Return"), (succ, "success", "Success␣rate")
        ]:
54      plt.figure(figsize=(7, 4))
55      plt.plot(range(W, N_EP + 1), ma(np.array(data)))
56      plt.xlabel("Episode"); plt.ylabel(f"{ylabel}␣(moving␣avg,␣window␣{W})")
57      plt.grid(alpha=0.3); plt.tight_layout()
58      plt.savefig(os.path.join(OUT, f"p1_{name}.png"), dpi=200); plt.close()
59
60  # ---- Greedy evaluation (ties broken randomly) ----
61  greedy = lambda i, s: int(rng.choice(np.flatnonzero(Q[i, s] == Q[i, s].max())))
62  print(f"S={S},␣k={k},␣g1={G1},␣g2={G2},␣last-{W}␣success={np.mean(succ[-W:]):.2f}")
63  for e in range(1, 4):
64      p1, p2, G = (1, 1), (5, 5), 0
65      print(f"\nEpisode␣{e}\nt=␣0␣|␣A1␣{p1}␣|␣A2␣{p2}")
66      for t in range(1, H + 1):
67          s = idx(p1, p2)
68          a1, a2 = greedy(0, s), greedy(1, s)
69          p1, p2, r, done = step(p1, p2, a1, a2)
70          G += r
71          print(f"t={t:2d}␣|␣A1␣{p1}␣{NAMES[a1]:5s}␣|␣A2␣{p2}␣{NAMES[a2]:5s}␣|␣r={r}")
72          if done:
73              break
74      print(f"Return={G},␣success={done}")
```

Problem 2:
Distributed MPC Feasibility with Competing STL Specifications (25 pts)

Consider a swarm of four mobile robots moving in a two-dimensional plane. The robots are indexed by $V = \{1, 2, 3, 4\}$. The communication topology is described by the cycle graph $G = (V, E)$ with $E = \{(1, 2), (2, 3), (3, 4), (4, 1)\}$. For each robot $i \in V$, let $\mathcal{N}_i$ denote its set of neighbours. The position of robot i at discrete time k is denoted by $p_i[k] \in \mathbb{R}^2$, and its dynamics are

$$p_i[k+1] = p_i[k] + u_i[k],$$

where $u_i[k] \in \mathbb{R}^2$ is the control input and it satisfies $\|u_i[k]\|_\infty \leq 0.5$. The initial positions are

$$p_1[0] = \begin{bmatrix} 0 \\ 0 \end{bmatrix},\ p_2[0] = \begin{bmatrix} 1 \\ 0 \end{bmatrix},\ p_3[0] = \begin{bmatrix} 1 \\ 1 \end{bmatrix},\ p_4[0] = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

So, the robots initially form a unit square. Define the desired relative displacement $\forall (i, j) \in E$ as $d_{ij} = p_i[0] - p_j[0]$.

Use the prediction horizon $H = 6$. At time k, Robot i receives predicted trajectories $\hat{p}_j[k + \ell|k]$, $j \in \mathcal{N}_i$, $\ell = 1, \ldots, H$ from its neighbours. Robot i optimizes only its own future control sequence

$$\mathbf{u}_i[k] = \begin{bmatrix} u_i[k|k] \\ u_i[k+1|k] \\ \vdots \\ u_i[k+H-1|k] \end{bmatrix}.$$

Consider the local distributed MPC objective

$$J_i = \sum_{\ell=1}^{H} \sum_{j \in \mathcal{N}_i} q \, \|(p_i[k+\ell|k] - \hat{p}_j[k+\ell|k]) - d_{ij}\|_2^2 + \sum_{\ell=0}^{H-1} r \, \|u_i[k+\ell|k]\|_2^2,$$

where $q > 0$, $r > 0$. Thus, each robot attempts to preserve the initial relative formation while minimizing control effort. Define the neighbour-formation predicate

$$F_{ij} := \|(p_i - p_j) - d_{ij}\|_\infty \leq 0.2.$$

Candidate STL Specification ϕ_1:

- throughout the prediction horizon, every pair of neighbouring robots must remain close to their original relative displacement;
- sometime within the prediction horizon, all robots must reach the original square formation translated upward by 2 units.

Candidate STL Specification ϕ_2:

- throughout the prediction horizon, every pair of neighbouring robots must again remain close to their original relative displacement;
- sometime within the prediction horizon, the robots must occupy a rectangular formation whose width is larger than the original square.

Define the target positions for specification 1 and 2 respectively as

$$g_i^{(1)} = p_i[0] + \begin{bmatrix} 0 \\ 2 \end{bmatrix},\ i = 1, \ldots, 4, \qquad g_1^{(2)} = \begin{bmatrix} 0 \\ 2 \end{bmatrix},\ g_2^{(2)} = \begin{bmatrix} 2 \\ 2 \end{bmatrix},\ g_3^{(2)} = \begin{bmatrix} 2 \\ 3 \end{bmatrix},\ g_4^{(2)} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}.$$

The corresponding STL specification are

$$\phi_1 = \Box_{[0,H]}\Big(\bigwedge_{(i,j)\in E} F_{ij} \Big) \wedge \Diamond_{[0,H]}\Big(\bigwedge_{i=1}^{4} \|p_i - g_i^{(1)}\|_\infty \le 0.1 \Big),$$

$$\phi_2 = \Box_{[0,H]}\Big(\bigwedge_{(i,j)\in E} F_{ij} \Big) \wedge \Diamond_{[0,H]}\Big(\bigwedge_{i=1}^{4} \|p_i - g_i^{(2)}\|_\infty \le 0.1 \Big).$$

Q1. Formulate the local distributed MPC optimization problem solved by Robot i. Clearly identify what are the local decision variables; the quantities known locally; and the predicted quantities received from neighbouring robots. Explain why, for fixed neighbour-predicted trajectories, the baseline local MPC problem is a convex optimization problem.

Q2. For the STL specification ϕ_1, translate the "always" formation-maintenance requirement into constraints on the predicted trajectories.

Q3. For the STL specification ϕ_2, translate the corresponding "always" formation-maintenance requirement into constraints on the predicted trajectories.

Q4. Determine which of the two STL specifications ϕ_1 or ϕ_2 results in a feasible distributed MPC problem at time $k = 0$. Your answer must be supported by a mathematical argument. A statement such as "a numerical solver reports feasible/infeasible" is not sufficient.

Q5. For the feasible STL specification, construct at least one explicit set of control sequences $\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3, \mathbf{u}_4$ that demonstrates feasibility. Verify that all input constraints are satisfied; the "always" STL requirement is satisfied throughout the prediction horizon; and the "eventually" STL requirement is satisfied at some point within the prediction horizon.

Solution:

Notation. $d_{12} = (-1, 0)^\top$, $d_{23} = (0, -1)^\top$, $d_{34} = (1, 0)^\top$, $d_{41} = (0, 1)^\top$, and $d_{ji} = -d_{ij}$. Neighbours: $\mathcal{N}_1 = \{2, 4\}$, $\mathcal{N}_2 = \{1, 3\}$, $\mathcal{N}_3 = \{2, 4\}$, $\mathcal{N}_4 = \{1, 3\}$.

Q1. Local MPC problem of robot i

- **Decision variables:** $\mathbf{u}_i = (u_i[k|k], \dots, u_i[k + H - 1|k]) \in \mathbb{R}^{2H}$.
- **Known locally:** $p_i[k]$, d_{ij}, weights q, r, the dynamics and the bound 0.5.
- **Received:** $\hat{p}_j[k + \ell|k]$ for $j \in \mathcal{N}_i$, $\ell = 1, \dots, H$.

$$\begin{aligned}
\min_{\mathbf{u}_i} \quad & J_i = \sum_{\ell=1}^{H} \sum_{j\in\mathcal{N}_i} q \, \|p_i[k + \ell|k] - \hat{p}_j[k + \ell|k] - d_{ij}\|_2^2 + \sum_{\ell=0}^{H-1} r \|u_i[k + \ell|k]\|_2^2 \\
\text{s.t.} \quad & p_i[k + \ell|k] = p_i[k] + \sum_{m=0}^{\ell-1} u_i[k + m|k], \quad \ell = 1, \dots, H, \\
& \|u_i[k + \ell|k]\|_\infty \le 0.5, \quad \ell = 0, \dots, H - 1.
\end{aligned}$$

Convexity. For fixed $\hat{p}_j$, every predicted position is an *affine* function of $\mathbf{u}_i$. Each cost term is a squared Euclidean norm of an affine function, hence convex, and a positive-weighted sum of convex functions is convex. Moreover $J_i = \mathbf{u}_i^\top \mathbf{H} \mathbf{u}_i + \mathbf{f}^\top \mathbf{u}_i + c$ with $\mathbf{H} \succeq rI \succ 0$, so it is strictly convex. The input constraints are linear (a box), so the feasible set is a convex polytope. The problem is therefore a convex quadratic program.

Q2. "Always" constraint for ϕ_1

$\Box_{[0,H]} \bigwedge_{(i,j)\in E} F_{ij}$ requires, for all $\ell = 0, \dots, H$ and all edges $(i, j) \in E$,

$$-0.2 \cdot \mathbf{1} \;\le\; p_i[k + \ell|k] - p_j[k + \ell|k] - d_{ij} \;\le\; 0.2 \cdot \mathbf{1}.$$

Robot i enforces this against the neighbour's received prediction, with $p_i[k+\ell|k] = p_i[k] + \sum_{m<\ell} u_i[k+m|k]$ (four linear inequalities per neighbour and step; for $\ell = 0$ it holds trivially since $p_i[0] - p_j[0] = d_{ij}$). Written out:

$$(1,2) : p_1 - p_2 \in (-1,0)^\top + [-0.2, 0.2]^2, \qquad (2,3) : p_2 - p_3 \in (0,-1)^\top + [-0.2, 0.2]^2,$$
$$(3,4) : p_3 - p_4 \in (1,0)^\top + [-0.2, 0.2]^2, \qquad (4,1) : p_4 - p_1 \in (0,1)^\top + [-0.2, 0.2]^2.$$

Q3. "Always" constraint for ϕ_2

The "always" part of ϕ_2 is identical to that of ϕ_1, so it gives exactly the same linear constraints as in Q2. The two specifications differ only in the "eventually" target.

Q4. Feasibility

ϕ_2 is infeasible. Suppose a trajectory satisfies ϕ_2. Then at some $\ell^\star \in [0, H]$, $\|p_i[\ell^\star] - g_i^{(2)}\|_\infty \le 0.1$ for all i, and by the "always" part F_{12} also holds at $\ell^\star$. Consider edge $(1,2)$, first component:

$$\left|(p_1 - p_2)_x - (g_1^{(2)} - g_2^{(2)})_x\right| \le 0.2 \;\Rightarrow\; (p_1 - p_2)_x \in [-2.2, -1.8],$$

since $(g_1^{(2)} - g_2^{(2)})_x = 0 - 2 = -2$. But F_{12} requires $(p_1 - p_2)_x \in [-1.2, -0.8]$ (as $(d_{12})_x = -1$). The two intervals are disjoint, a contradiction. (Edge $(3,4)$ gives the same conflict: $[1.8, 2.2]$ versus $[0.8, 1.2]$.) Hence no control sequence, whatever the input bounds, satisfies ϕ_2: the target width 2 is incompatible with the required width 1 ± 0.2.

ϕ_1 is feasible. The target $g_i^{(1)} = p_i[0] + (0,2)^\top$ is the whole formation translated rigidly, so $g_i^{(1)} - g_j^{(1)} = d_{ij}$ and the target satisfies the "always" constraint exactly. Reaching it requires a vertical displacement of 2, and the input bound allows $6 \times 0.5 = 3 \ge 2$ within $H = 6$ steps (at least $2/0.5 = 4$ steps are needed). An explicit construction is given in Q5.

Q5. Explicit feasible controls for ϕ_1

For every robot $i = 1, \dots, 4$:

$$u_i[\ell] = \begin{cases} (0,\, 0.5)^\top & \ell = 0, 1, 2, 3, \\ (0,\, 0)^\top & \ell = 4, 5. \end{cases}$$

- **Input constraint:** $\|u_i[\ell]\|_\infty \in \{0, 0.5\} \le 0.5$.
- **Always:** all robots receive the same input, so $p_i[\ell] = p_i[0] + (0,\, 0.5\min(\ell, 4))^\top$ and $p_i[\ell] - p_j[\ell] = p_i[0] - p_j[0] = d_{ij}$. The formation error is $0 \le 0.2$ for all $\ell = 0, \dots, 6$.
- **Eventually:** at $\ell = 4$, $p_i[4] = p_i[0] + (0,2)^\top = g_i^{(1)}$, so $\|p_i[4] - g_i^{(1)}\|_\infty = 0 \le 0.1$ (also for $\ell = 5, 6$).

Robot	$\ell = 0$	1	2	3	4	5	6
1	$(0,0)$	$(0,0.5)$	$(0,1)$	$(0,1.5)$	$(0,2)$	$(0,2)$	$(0,2)$
2	$(1,0)$	$(1,0.5)$	$(1,1)$	$(1,1.5)$	$(1,2)$	$(1,2)$	$(1,2)$
3	$(1,1)$	$(1,1.5)$	$(1,2)$	$(1,2.5)$	$(1,3)$	$(1,3)$	$(1,3)$
4	$(0,1)$	$(0,1.5)$	$(0,2)$	$(0,2.5)$	$(0,3)$	$(0,3)$	$(0,3)$

Table 2: Predicted positions for ϕ_1. The script reports input constraint: True, always: True, eventually at $\ell = 4, 5, 6$.

Code (numerical check of Q4–Q5)

```python
import numpy as np

H = 6
p0 = np.array([[0, 0], [1, 0], [1, 1], [0, 1]], float)
E = [(0, 1), (1, 2), (2, 3), (3, 0)]
err = lambda P: [np.abs(P[i] - P[j] - (p0[i] - p0[j])).max() for i, j in E]   # formation errors

g1 = p0 + [0, 2]
g2 = np.array([[0, 2], [2, 2], [2, 3], [0, 3]], float)

# phi_2: the target itself violates the formation constraint (limit 0.2)
print("phi_2 edge errors at target:", err(g2))

# phi_1: all robots move up 0.5 for 4 steps, then stop
u = np.zeros((4, H, 2))
u[:, :4, 1] = 0.5
P = p0[:, None] + np.concatenate([np.zeros((4, 1, 2)), np.cumsum(u, 1)], 1)   # shape (4, H+1, 2)

for i in range(4):
    print(f"robot {i+1}:", P[i].tolist())
print("input ok :", np.abs(u).max() <= 0.5)
print("always ok :", all(max(err(P[:, l])) <= 0.2 + 1e-9 for l in range(H + 1)))
print("eventually at l =", [l for l in range(H + 1) if np.abs(P[:, l] - g1).max() <= 0.1])
```

Problem 3:
The Chief Minister's AI Policy Simulator (50 pts)

The Hon'ble Chief Minister of Telangana, proposes to introduce a new public welfare scheme for the people of Telangana. Before launching the scheme, the Chief Minister approaches you, an AI expert, with the following question:

CM's request to you. *My government is planning to introduce a new welfare scheme. I expect the scheme to increase public participation, but this may also increase the workload of government departments. Can you build an AI-based simulation of how the scheme will operate over the next 365 days, identify administrative bottlenecks, and recommend whether the scheme should be launched? Can the government departments learn to cooperate and improve their collective performance?*

Your task is to construct a stochastic model of government administration and subsequently formulate the problem as a cooperative Multi-Agent Reinforcement Learning (MARL) problem. The objective is to evaluate whether decentralized departmental decision-making can improve government service delivery and public satisfaction.

Disclaimer. The proposed scheme, administrative workflow, and numerical parameters are hypothetical. The exercise does not attempt to reproduce the actual organizational hierarchy of the Telangana government or predict the political performance of any existing scheme.

Consider a government administration consisting of N departments, represented by the set $V = \{D_1, D_2, \ldots, D_N\}$. The departments form a directed hierarchical network $G = (V, E)$, where $E = \{(D_i, D_{i+1}) \mid i = 1, \ldots, N-1\}$. Every application submitted by a citizen must pass sequentially through all departments, starting at D_1 and terminating at D_N. An application can move to department D_{i+1} only after its processing at department D_i is complete. For this assignment, take $N = 5$ which are

1. D_1: Application Intake
2. D_2: Eligibility Verification
3. D_3: Technical Assessment
4. D_4: Financial Assessment
5. D_5: Final Approval

Each department is equipped with processing officers who handle applications according to a First-In-First-Out (FIFO) discipline. The working duration is $H = 480$ minutes per day. The processing requirements are given below.

Department	Time per application (minutes)	Initial officers
D_1	20	3
D_2	30	3
D_3	60	3
D_4	40	3
D_5	30	3

An officer can process only one application at a time. Different officers may process different applications simultaneously. An application may pass through several departments within the same working day. If an application cannot complete all processing stages by the end of the working day, it resumes from its current processing stage on the following day. Any partially completed processing effort must be preserved; the application does not restart from D_1. There is no overtime, and unprocessed applications remain in their departmental queues.

Let $d \in \{1, \ldots, 365\}$ denote the day index. The number of new applications received on day d is a random variable A_d satisfying $A_d \sim \text{Poisson}(\lambda_d)$, where λ_d is the expected daily demand. Initially, assume $\lambda_d = 16\theta$, $d = 1, \ldots, 365$. The parameter θ represents the impact of the new scheme on public applications. Consider the following four scenarios:

Scenario	θ	Interpretation
S1	0.75	Reduced demand
S2	1.00	Normal demand
S3	1.25	Increased demand
S4	1.50	High demand

(a) Simulate the arrival and processing of applications over 365 days for all four scenarios.
(b) Track each application individually, including its arrival day, current department, remaining processing requirement, and completion day.
(c) Let $Q_i(d)$ denote the number of applications present at department D_i at the end of day d. Plot $Q_i(d)$ for all five departments.
(d) Let B_d denote the total number of unfinished applications at the end of day d. Plot the evolution of B_d for all four scenarios.
(e) Investigate whether an increase in application arrivals necessarily results in a corresponding increase in completed applications.

Solution:

(a) Simulation. The simulator (code at the end of Problem 3) runs 365 days for each scenario:

- Each day $A_d \sim \text{Poisson}(\lambda_d)$ applications arrive at D_1 at minute 0.
- In each department, applications are served FIFO by m_i officers working in parallel for 480 minutes. Departments are processed in the order $D_1 \to D_5$ within a day, so an application can pass through several departments the same day.
- No overtime: work stops at minute 480. Partially served applications keep their remaining time and resume at the same department the next day.

Daily capacity with 3 officers, $3 \cdot 480/T_i$, is 72, 48, **24**, 36, 48 applications for $D_1, \ldots, D_5$. D_3 is the bottleneck; its utilisation at mean demand 16θ is $\rho_3 = 16\theta/24 = 0.50, 0.67, 0.83, 1.00$ for S1–S4.

(b) Per-application tracking. For every application the simulator stores its arrival day, its current department (stage), its remaining service minutes at that department, and its completion day (set when it leaves D_5). $Q_i(d)$ and B_d are obtained from these records at the end of each day.

(c) Queue lengths $Q_i(d)$.

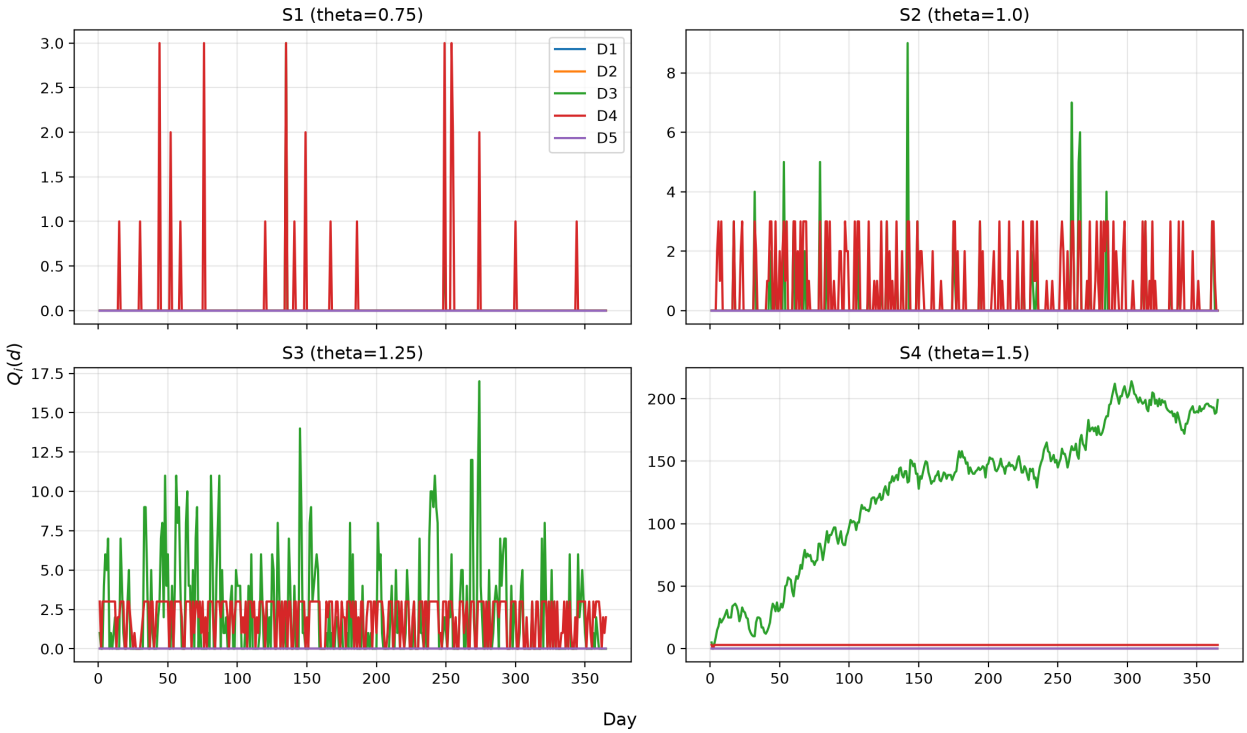


Figure 2: Queue length $Q_i(d)$ at each department, scenarios S1–S4 (constant demand, fixed staffing).

(d) Unfinished applications B_d.

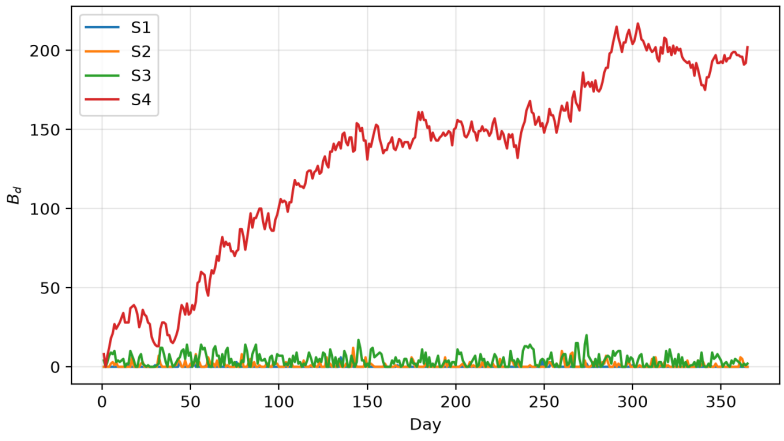


Figure 3: Evolution of B_d for S1–S4 (constant demand, fixed staffing).

(e) Do more arrivals mean proportionally more completions? Mean over 20 seeds:

	S1	S2	S3	S4
Arrivals	4382	5843	7323	8775
Completed	4382	5843	7320	8697
Unfinished at day 365	0	0	3	79

No. Completions track arrivals only while every department has spare capacity. Throughput is capped by D_3 at 24 applications/day (8760/year); at S4 the demand equals this cap, completions fall behind arrivals (79 unfinished) and the gap grows with demand. More arrivals therefore do not yield proportionally more completions, and, as shown next, they lower the reward.

Problem 3 (cont.):
Assume that the citizens evaluate the government based on the timeliness of service delivery. Suppose application j arrives on day a_j and completes processing on day c_j. If the application is completed on its arrival day, the government receives a reward of $+5$. If an application remains incomplete at the end of a day, the government incurs a penalty of -10 for that pending application. The daily government reward is therefore

$$r_d = 5\, C_d^{\text{same}} - 10\, B_d, \tag{1}$$

where C_d^{same} denotes the number of applications that arrive and are completed on day d. An application completed after its arrival day receives no completion bonus. Its delay penalties have already been accumulated during the days on which it remained unfinished. The cumulative government reward is $R_d = \sum_{k=1}^{d} r_k$.

(a) Plot R_d over 365 days for all four scenarios (S1 – S4).
(b) Explain the relationship between processing bottlenecks, pending applications, and cumulative government reward.

Solution:

(a) Cumulative reward.

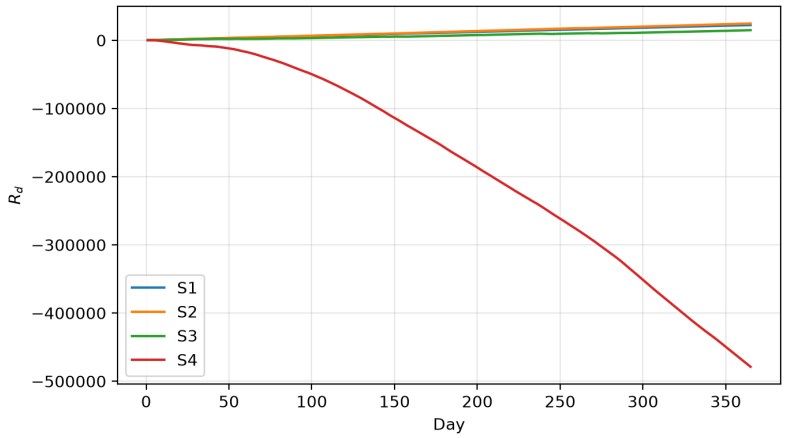


Figure 4: Cumulative reward R_d for S1–S4 (constant demand, fixed staffing).

(b) Bottlenecks, pending applications and reward. The reward $r_d = 5C_d^{\text{same}} - 10B_d$ rewards only same-day completions, while every pending application costs 10 per day. A bottleneck (D_3) slows the whole pipeline, so more applications are still unfinished at the end of the day ($B_d \uparrow$) and fewer finish on their arrival day ($C_d^{\text{same}} \downarrow$). Both effects reduce r_d. If the backlog persists or grows, r_d stays negative and R_d decreases steadily, whereas an uncongested system accumulates positive reward from same-day completions.

Problem 3 (cont.):
Cooperative MARL Formulation: The Chief Minister recognizes that the departmental staffing configuration need not remain fixed. Departments may cooperate by temporarily sharing processing officers with their immediate neighbours. Instead of prescribing a centralized staffing strategy, the government allows each department to learn its own resource-allocation policy. The five departments now become five cooperative RL agents. The set of agents is $\mathcal{N} = \{1, 2, 3, 4, 5\}$. Agent i corresponds to department D_i. All agents cooperate to maximize the cumulative government reward. At the beginning of day d, let $Q_i^{\text{start}}(d)$ denote the number of applications present at department i, including newly arriving applications at department D_1. Each department observes its own queue, the queues of

its immediate neighbours, and the public satisfaction from the previous day. For interior departments,

$$o_i(d) = \big(Q_{i-1}^{\text{start}}(d),\, Q_i^{\text{start}}(d),\, Q_{i+1}^{\text{start}}(d),\, q_{d-1}\big).$$

The first and last departments observe only their existing neighbours. The quantity $q_d \in [-1, 1]$ represents public satisfaction, defined subsequently. Each agent makes its decision using its own local observation. At the beginning of every day, each agent selects an action

$$a_i(d) = \begin{cases} -1, & \text{lend one officer to } D_{i-1}, \\ 0, & \text{retain all officers}, \\ +1, & \text{lend one officer to } D_{i+1}. \end{cases}$$

Boundary departments cannot transfer officers to non-existent neighbours. Thus, their admissible action sets are $\mathcal{A}_1 = \{0, +1\}$, and $\mathcal{A}_5 = \{-1, 0\}$ respectively. For $i = 2, 3, 4$, $\mathcal{A}_i = \{-1, 0, +1\}$.

Assume that officers are temporarily transferable between neighbouring departments and are qualified to perform the corresponding tasks. An officer transferred to another department works there for the entire day and returns to the original department the following morning. A department may lend at most one of its original three officers per day. Borrowed officers cannot be transferred onward to another department on the same day. Let $m_i(d)$ denote the effective number of officers available at department i on day d. Since officers are only redistributed, $\sum_{i=1}^{5} m_i(d) = 15$. All five agents receive the same reward r_d given by (1). The cooperative objective is

$$\max_{\pi_1,\ldots,\pi_5} \mathbb{E}\Big[\sum_{d=1}^{365} \gamma^{d-1} r_d\Big],$$

where π_i denotes the policy of agent i and $\gamma \in (0, 1]$ is the discount factor. The underlying environment state includes all pending applications, their processing stages, remaining service requirements, application ages, and public demand variables. Agents do not observe this complete state. They operate using their local observations.

(a) Explain why this formulation is a cooperative MARL problem rather than a single-agent reinforcement learning problem.
(b) Identify the agents, observations, actions, environment state, transition mechanism, and shared reward.
(c) Consider a situation where D_3 is heavily congested while D_2 has very few pending applications. Explain how officer sharing could improve the overall processing performance.
(d) Can transferring an officer from one department to another ever worsen the global reward? Justify your answer.

Solution:

(a) Why cooperative MARL. Five decision-makers each control only their own lending action, from a local observation, and all share one reward. The staffing $m_i(d)$ depends on the *joint* action, and the reward cannot be split into per-department parts, because the outcome (same-day completions, backlog) depends on all departments in the chain. A single-agent formulation would need one controller with the full state and the joint action space $2 \cdot 3 \cdot 3 \cdot 3 \cdot 2 = 108$. Here the information is distributed and observations are partial, so it is a cooperative (Dec-POMDP) multi-agent problem.

(b) Components.

- **Agents:** $\mathcal{N} = \{1, \ldots, 5\}$, agent i = department D_i.
- **State:** all pending applications (stage, remaining service time, age, arrival day), plus q_{d-1}, p_{d-1} and the demand level λ_d.
- **Observation:** $o_i(d) = (Q_{i-1}^{\text{start}}, Q_i^{\text{start}}, Q_{i+1}^{\text{start}}, q_{d-1})$; $o_1 = (Q_1, Q_2, q_{d-1})$ and $o_5 = (Q_4, Q_5, q_{d-1})$.
- **Actions:** $a_i \in \{-1, 0, +1\}$ (lend to D_{i-1}, keep, lend to D_{i+1}), with $\mathcal{A}_1 = \{0, +1\}$, $\mathcal{A}_5 = \{-1, 0\}$.
- **Staffing:** $m_i = 3 - \mathbb{1}[a_i \neq 0] + \mathbb{1}[a_{i-1} = +1] + \mathbb{1}[a_{i+1} = -1]$, with $a_0 = a_6 = 0$, so $\sum_i m_i = 15$.

- **Transition:** draw $A_d \sim \text{Poisson}(\lambda_d)$; process FIFO with m_i officers; move finished applications to the next department; update p_d, q_d, $\lambda_{d+1} = 16\theta(1 + 0.5q_d)$. The only randomness is the arrival process.
- **Shared reward:** $r_d = 5C_d^{\text{same}} - 10B_d$ for all agents; objective $\max_{\pi_1,\ldots,\pi_5} \mathbb{E}\big[\sum_{d=1}^{365} \gamma^{d-1} r_d\big]$.

(c) Congested D_3, idle D_2. D_2 lends one officer to D_3 ($a_2 = +1$): $m_3 = 4$, $m_2 = 2$. The capacity of D_3 rises from 24 to 32 applications/day, while D_2 still handles 32, well above its small queue. The idle officer is converted into useful work at the bottleneck, so the D_3 queue drains faster, applications reach D_4, D_5 earlier, more finish on their arrival day, and fewer remain pending, which raises r_d.

(d) Can a transfer worsen the global reward? Yes. A transfer is beneficial only if the lender has slack and the receiver has work. It hurts when (i) the lender is itself the bottleneck, e.g. D_3 lends to D_4: capacity falls from 24 to 16, below a demand of ≈ 20/day, so the backlog grows; (ii) the receiver has no queue, so the lender's capacity is lost with no gain; (iii) help is given upstream of the bottleneck, which only moves the queue into D_3; (iv) agents acting on local observations lend simultaneously and over-correct (e.g. D_2 and D_4 both send an officer to D_3 while both are loaded).

Problem 3 (cont.):
Let S_d denote the number of applications received on day d that are successfully completed on the same day. Define the on-time completion fraction as

$$p_d = \frac{S_d}{A_d}, \qquad A_d > 0.$$

If $A_d = 0$, use $p_d = p_{d-1}$, with $p_0 = 0.5$. Public satisfaction on day d, denoted by q_d evolves according to

$$q_d = 0.8\, q_{d-1} + 0.2\,(2p_d - 1), \qquad \text{with } q_0 = 0.$$

The expected number of applications on the following day and the actual arrivals are generated respectively by

$$\lambda_{d+1} = 16\theta\,(1 + 0.5\, q_d), \qquad A_{d+1} \sim \text{Poisson}(\lambda_{d+1}).$$

Therefore, better service improves public satisfaction and potentially increases future demand. Conversely, delayed service reduces public satisfaction and discourages future applications.

(a) Integrate the public satisfaction model into the government simulation.
(b) Explain why an improvement in government service delivery may eventually lead to increased departmental workloads.
(c) Explain how the actions of a single department can indirectly influence future application arrivals across the entire government network.
(d) Is it possible for a highly popular scheme to initially perform well and subsequently fail because of administrative congestion?

Solution:

(a) Integration. The recursion $p_d = S_d/A_d$, $q_d = 0.8q_{d-1} + 0.2(2p_d - 1)$, $\lambda_{d+1} = 16\theta(1 + 0.5q_d)$ is implemented inside the simulator (the `feedback=True` option); arrivals on day $d + 1$ are drawn from Poisson(λ_{d+1}). Results with feedback (fixed staffing):

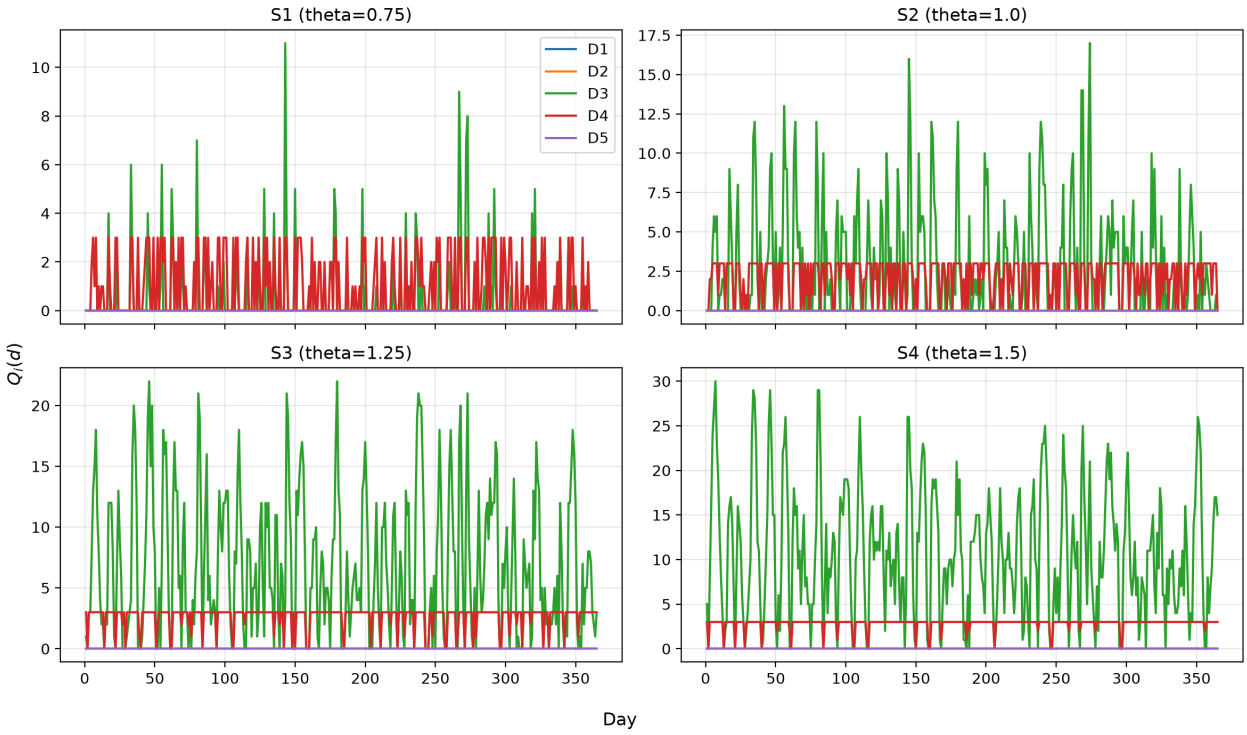


Figure 5: Queue length $Q_i(d)$ with satisfaction feedback, S1–S4.

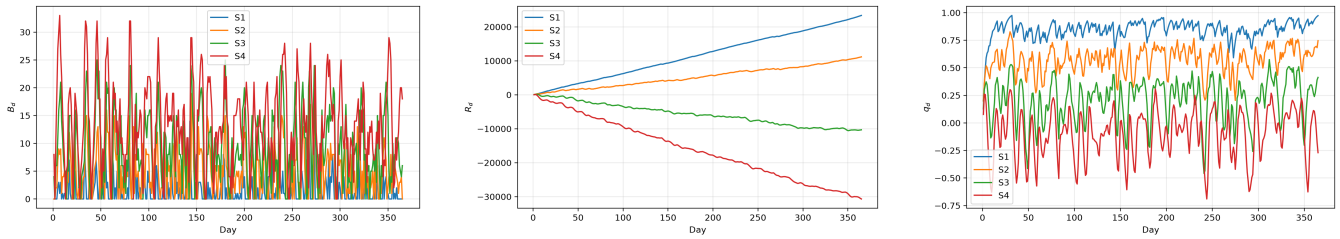


Figure 6: With feedback: backlog B_d (left), cumulative reward R_d (middle), public satisfaction q_d (right).

(b) Better service raises workload. Good service raises p_d and hence q_d, which raises λ_{d+1} by up to 50% (at $q = 1$). In S1 the final satisfaction $q_{365} = 0.86$ lifts the demand from the nominal 12 to about 17 applications/day (6269 received against the nominal 4380), which increases the utilisation of every department.

(c) Indirect influence of one department. Every application passes through all departments, so a delay or a faster service at one department changes the end-of-day backlog and the number of same-day completions C_d^{same} of the whole chain. These determine p_d and the *single, common* q_d, which sets λ_{d+1}, the arrivals at D_1, and hence the load on all departments. A lending decision at one department thus changes future demand for the entire network.

(d) Popular scheme failing through congestion. Yes. A scheme can start well (empty queues, $q \uparrow$), which raises demand until the effective load exceeds the capacity of D_3 (24/day). Then B_d grows, p_d and q_d fall, and the reward turns negative; demand drops only afterwards, once the damage to satisfaction is done. In the simulation, the average arrival rate reaches 22.7/day (S3) and 23.4/day (S4) against the capacity of 24, and q_{365} falls to 0.28 and -0.14 respectively.

Problem 3 (cont.):
The Chief Minister now requests a final assessment of the proposed scheme based on the performance of the learned cooperative departmental policies. Evaluate the following administrative system

where every department retains its original three officers on every day. System must use the above departmental processing requirements and public demand feedback model. Test the system under all four demand scenarios S1–S4. For every scenario and administrative system, conduct at least 20 independent 365-day evaluation simulations using different random seeds. Report the following:

(a) Total number of applications received during the year.
(b) Total number of completed applications.
(c) Fraction of applications completed on the same day.
(d) Average completion delay among completed applications.
(e) Number of unfinished applications at the end of day 365.
(f) Cumulative government reward R_{365}.
(g) Final public satisfaction q_{365}.
(h) Average effective staffing level at each department over the year.

Based on your evaluation, should the Chief Minister launch the scheme under the existing fixed-staffing system? Your recommendation must address administrative feasibility, public satisfaction, resource allocation, and the risks associated with uncertain future demand.

Solution:

(a)–(h) Results. Fixed staffing (3 officers per department), public-satisfaction feedback on; 20 independent 365-day runs per scenario (different seeds); mean ± standard deviation.

Metric	S1 ($\theta = 0.75$)	S2 ($\theta = 1.00$)	S3 ($\theta = 1.25$)	S4 ($\theta = 1.50$)
(a) Applications received	6268.8 ± 52.7	7596.4 ± 51.6	8265.5 ± 38.5	8524.1 ± 36.3
(b) Completed	6266.5 ± 53.4	7592.6 ± 52.2	8255.8 ± 39.2	8507.7 ± 32.3
(c) Same-day fraction	0.92 ± 0.00	0.78 ± 0.01	0.62 ± 0.01	0.46 ± 0.01
(d) Avg. delay (days)	0.08 ± 0.00	0.22 ± 0.01	0.38 ± 0.01	0.55 ± 0.01
(e) Unfinished at day 365	2.3 ± 3.1	3.8 ± 4.2	9.8 ± 4.5	16.4 ± 9.0
(f) Cumulative reward R_{365}	23917 ± 329	13009 ± 814	-6313 ± 1160	-27377 ± 1094
(g) Final satisfaction q_{365}	0.86 ± 0.07	0.65 ± 0.08	0.28 ± 0.18	-0.14 ± 0.19
(h) Avg. staffing $D_1, \ldots, D_5$	3.0, 3.0, 3.0, 3.0, 3.0 (fixed, all scenarios)			

Table 3: Fixed-staffing evaluation. (c) is the number of same-day completions divided by the number of applications received.

Recommendation

Launch only if demand stays at or below normal ($\theta \leq 1$); do not launch with fixed staffing if high demand is expected ($\theta \geq 1.25$).

- **Administrative feasibility:** the system is stable in S1–S2 but runs close to its limit as demand grows. With feedback, the average arrival rate is 17.2, 20.8, 22.7, 23.4/day in S1–S4, i.e. D_3 utilisation 0.72, 0.87, 0.94, 0.97. Almost all applications are eventually completed in every scenario, but their timeliness collapses: the same-day fraction falls from 0.92 to 0.46 and the delay grows from 0.08 to 0.55 days.
- **Public satisfaction:** high in S1–S2 (0.86, 0.65), weak in S3 (0.28) and negative in S4 (-0.14). The reward follows the same pattern: +23.9k, +13.0k, -6.3k, -27.4k.
- **Resource allocation:** the equal 3-officer allocation is inefficient. D_3 is the only bottleneck, while D_1, D_2, D_4, D_5 have capacities $72, 48, 36, 48$ against at most ≈ 23 arrivals/day. Moving officers towards D_3 (or adding capacity there) would remove the constraint; this is what the cooperative sharing scheme is designed to do.
- **Demand uncertainty:** the outcome is sensitive to θ: a rise of 25% from normal demand ($\theta = 1 \to 1.25$) changes R_{365} from +13.0k to -6.3k. Because success increases demand (feedback), and demand is hard to predict, a fixed staffing level carries a real risk of congestion; the launch should come with monitoring of the D_3 queue and a plan for flexible reallocation.

Scope note. Only the fixed-staffing system was simulated, as specified in the final evaluation; the cooperative officer-sharing system is formulated in Part (b) of the MARL section but no policy was trained.

Code (Problem 3)

```
import os
import heapq
import numpy as np
import matplotlib.pyplot as plt

OUT = os.path.dirname(os.path.abspath(__file__))
T = [20, 30, 60, 40, 30]            # service minutes at D1..D5
M = [3, 3, 3, 3, 3]                 # officers per department (fixed staffing)
DAY, DAYS, N_SEEDS = 480, 365, 20
SCEN = {"S1": 0.75, "S2": 1.0, "S3": 1.25, "S4": 1.5}
DEPTS = ["D1", "D2", "D3", "D4", "D5"]
days = np.arange(1, DAYS + 1)


def simulate(theta, seed, feedback):
    rng = np.random.default_rng(seed)
    arr, comp, rem, stage = [], [], [], []              # per-application records
    qs = [[] for _ in range(5)]                         # FIFO queue of application ids
        per dept
    q, p_prev = 0.0, 0.5
    Q, r_list, q_list, same = [], [], [], 0
    for d in range(1, DAYS + 1):
        A = rng.poisson(16 * theta * (1 + 0.5 * q) if feedback else 16 * theta)
        for _ in range(A):
            qs[0].append(len(arr)); arr.append(d); comp.append(0); rem.append(T[0]);
                stage.append(0)
        ready = [0] * len(arr)                          # minute at which an app becomes
             available today
        cs = 0
        for i in range(5):
            free, left = [0] * M[i], []                 # officer free-times (heap),
                apps left for tomorrow
            order = sorted(qs[i], key=lambda a: (ready[a], a))        # FIFO
            for n, a in enumerate(order):
                start = max(free[0], ready[a])
                if start >= DAY:                        # no overtime
                    left += order[n:]
                    break
                end = min(start + rem[a], DAY)
                heapq.heapreplace(free, end)
                rem[a] -= end - start                   # partial work is preserved
                if rem[a] > 0:
                    left.append(a)
                elif i == 4:
                    comp[a] = d
                    cs += arr[a] == d
                else:
                    stage[a], rem[a], ready[a] = i + 1, T[i + 1], end
                    qs[i + 1].append(a)
            qs[i] = left
        b = [len(x) for x in qs]
        p = cs / A if A else p_prev
        p_prev = p
        q = 0.8 * q + 0.2 * (2 * p - 1)
        Q.append(b); r_list.append(5 * cs - 10 * sum(b)); q_list.append(q); same += cs
    Q = np.array(Q)
    return dict(Q=Q, B=Q.sum(1), R=np.cumsum(r_list), q=np.array(q_list),
                arr=np.array(arr), comp=np.array(comp), same=same)

```

```python

def stats(r):
    done = r["comp"] > 0
    n = len(r["arr"])
    return [n, done.sum(), r["same"] / n, (r["comp"] - r["arr"])[done].mean(), r["B"
        ][-1], r["R"][-1], r["q"][-1]]


def make_plots(runs, tag, keys):
    fig, axs = plt.subplots(2, 2, figsize=(12, 7), sharex=True)
    for ax, (n, r) in zip(axs.flat, runs.items()):
        ax.plot(days, r["Q"]); ax.set_title(f"{n} (theta={SCEN[n]})"); ax.grid(alpha=0.3)
    axs[0, 0].legend(DEPTS); fig.supxlabel("Day"); fig.supylabel("$Q_i(d)$"); fig.
        tight_layout()
    fig.savefig(os.path.join(OUT, f"p3_{tag}_queues.png"), dpi=200); plt.close(fig)
    for key, label in keys:
        plt.figure(figsize=(7, 4))
        for n, r in runs.items():
            plt.plot(days, r[key], label=n)
        plt.xlabel("Day"); plt.ylabel(label); plt.legend(); plt.grid(alpha=0.3); plt.
            tight_layout()
        plt.savefig(os.path.join(OUT, f"p3_{tag}_{key}.png"), dpi=200); plt.close()


print("Daily capacity per dept:", [round(m * DAY / t, 1) for m, t in zip(M, T)])

# ---- Part 1: constant demand (no feedback) ----
static = {n: simulate(th, 0, False) for n, th in SCEN.items()}
make_plots(static, "static", [("B", "$B_d$"), ("R", "$R_d$")])

# ---- Part (e): do more arrivals mean more completions? ----
print("\nConstant demand, mean of", N_SEEDS, "seeds")
for n, th in SCEN.items():
    s = np.mean([stats(simulate(th, i, False)) for i in range(N_SEEDS)], axis=0)
    print(f"{n}: arrivals={s[0]:.0f}, completed={s[1]:.0f}, unfinished={s[4]:.0f}")

# ---- Part 2: with public-satisfaction feedback ----
fb = {n: simulate(th, 0, True) for n, th in SCEN.items()}
make_plots(fb, "feedback", [("B", "$B_d$"), ("R", "$R_d$"), ("q", "$q_d$")])

# ---- Final evaluation: fixed staffing + feedback, 20 seeds ----
names = ["Received", "Completed", "Same-day fraction", "Avg delay (days)", "Unfinished (
    day 365)", "R_365", "q_365"]
res = {n: np.array([stats(simulate(th, 1000 + i, True)) for i in range(N_SEEDS)]) for n,
    th in SCEN.items()}
print("\nFixed staffing, feedback ON, mean +- std over", N_SEEDS, "seeds")
print(f"{'':22s}" + "".join(f"{n:>22s}" for n in SCEN))
for j, name in enumerate(names):
    print(f"{name:22s}" + "".join(f"{res[n][:, j].mean():12.2f} +-{res[n][:, j].std(ddof
        =1):7.2f}" for n in SCEN))
print("Avg staffing per dept (fixed):", [float(m) for m in M])
```

AI usage declaration. An AI assistant was used to help write and debug the Python code and to clean up the structure of this report. All numerical results were generated by running the submitted code, and I reviewed the derivations and checked the results.